DSST389: Exam II — Reference Sheet

This sheet is attached to your exam. You do not need to memorize any of it.

Dates and datetimes.

- `pl.date(year, month, day)` — build a `Date` from three columns. `pl.datetime(y, m, d, h, m, s)` builds a `Datetime`.
- `c.col.str.to_datetime()` — parse a string. Needs no format string if the text is ISO-8601 (`YYYY-MM-DD HH:MM:SS`); otherwise pass one, e.g. `"%Y-%m-%dT%H:%M:%SZ"`.
- Components: `.dt.year()`, `.dt.month()`, `.dt.day()`, `.dt.hour()`, `.dt.ordinal_day()`, `.dt.quarter()`, `.dt.date()`, and `.dt.weekday()` (Monday = 1, Sunday = 7).
- `.dt.truncate("1h")` — round *down* to a unit, turning a timestamp into a bin you can group by. `.dt.round()` rounds to the nearest instead. Durations: `"1h"`, `"1d"`, `"30d"`, `"1mo"`.
- `.scale_x_date(date_breaks="1 month", date_labels="%Y-%m")` and `.scale_x_datetime(...)` label a time axis.

Time zones. A datetime is either *naive* (no zone) or *aware* (tagged with one). The two methods below look alike and do opposite things.

- `.dt.replace_time_zone("UTC")` — **attaches** a zone to a naive datetime. The clock reading does not change. This is a *claim* about what the data always meant. Calling it on an already-aware column moves the instant.
- `.dt.convert_time_zone("America/New_York")` — **re-expresses** an aware datetime in another zone. The clock reading *does* change. This is a *calculation*. Calling it on a naive column silently assumes UTC.
- Wikipedia reports every revision timestamp in UTC. Attach, then convert.

Durations. Subtracting two datetimes gives a `Duration`. Durations average exactly.

- `.dt.total_hours()` `.dt.total_days()` `.dt.total_minutes()` `.dt.total_seconds()`
- These **truncate toward zero**, and they do it to each value *before* any aggregation. A gap of 5h 35m becomes 5 hours, or 0 days. Choose a unit finer than the intervals you are measuring.

Window functions. An ordered function needs a `.sort()` in front of it to say what "previous" means, and an `.over()` to stop it running across the boundary between one group and the next.

- `c.size.shift(1).over(c.doc_id)` — the value one row back, within each group. `shift(-1)` looks forward; `shift(2)` reaches back two.
- `c.size.diff().over(c.doc_id)` — shorthand for `c.size - c.size.shift(1)`. The first row of each group is `null`.
- `c.size.rolling_mean(window_size=3)` — a window of **three rows**. Only correct if rows are evenly spaced.
- `c.size.rolling_mean_by(by=c.timestamp, window_size="30d")` — a window of **thirty days**, however many rows that is. Use this whenever observations are unevenly spaced in time. Also `rolling_sum_by`, `rolling_max_by`, `rolling_std_by`.
- A comparison against `null` is `null`, not `False`, and `.filter()` keeps only rows that are `True`.

Joins on time.

```
df.join_asof(other, left_on="timestamp", right_on="moment", by="doc_id",
             strategy="backward", tolerance=None)
```

- Matches each left row to the *nearest* right row rather than an equal one. Both tables must be sorted on the join key. `by=` forces an exact match on a group first, playing the same role that `.over()` plays for a window function.
- `strategy`: `"backward"` (the default) takes the most recent record at or before; `"forward"` takes the next at or after; `"nearest"` takes whichever is closer, and so can reach into the future. An asof join does not return `null` for want of a *nearby* match, however far it has to reach; `tolerance` caps the distance.
- `df.join_where(other, expr, ...)` — keep every pair of rows satisfying a filter expression. Columns of the right table carry a `_right` suffix.

The annotation table. `DSText.process(docs, nlp)` returns one row per token. The primary key is `doc_id`, `sid`, `tid` together.

- `sid` sentence number; `tid` token number **within the sentence**; `token` the text; `token_with_ws` the text plus trailing whitespace; `lemma` the normalized form (lowercased, singular, infinitive).
- `upos` the part of speech: `NOUN`, `VERB`, `ADJ`, `ADV`, `PROPN`, `DET`, `ADP`, `NUM`, `PUNCT`. `tag` is the fine-grained, English-specific version.

- `is_alpha`, `is_stop`, `is_punct` — Boolean flags. A number such as `1847` is *not* alphabetic.
- `dep` the grammatical relation (`nsubj`, `dobj`, `amod`, `det`, `prep`, `pobj`, `compound`, `ROOT`); `head_idx` the `tid` of the token it depends on, in the same sentence-level coordinate system as `tid`. The root points at itself.
- `ent_type` the entity type (`PERSON`, `ORG`, `GPE`, `DATE`, `WORK_OF_ART`), empty if none; `ent_iob` is `B` at the first token of an entity, `I` inside one, `O` outside any.
- `DSText.kwic(anno, keyword, max_num=10, window=5)` — print a keyword-in-context display. Prints; does not return a table.

N-grams and collocations. Build a bigram by shifting the lemma column up one within each sentence, then pairing.

```
next_token = c.lemma.shift(-1).over([c.doc_id, c.sid])
bigram     = c.lemma + " " + c.next_token
```

Pointwise mutual information compares a bigram's frequency against what independence would predict. $P(w)$ is a word's count over the total number of words; $P(w_1, w_2)$ is the bigram's count over the total number of bigrams. Filter to bigrams occurring at least five times before ranking, or the top of the list fills with pairs of rare words that happened to co-occur once.

$$\mathrm{PMI}(w_1, w_2) \;=\; \log \frac{P(w_1, w_2)}{P(w_1) \cdot P(w_2)}$$

TF-IDF. `DSText.compute_tfidf(anno, min_df=0.0, max_df=1.0)` returns one row per non-zero (document, lemma) pair. It keeps only alphabetic tokens.

- `tf_norm` is the term's share of the tokens *in its document*, not a raw count. `df` is the number of documents containing the term, and N is the total number of documents.
- `min_df` and `max_df` are **proportions** in $[0, 1]$, compared with `<=` and `>=`. `max_df=0.5` drops terms in *more than* half the documents; a term in exactly half survives. Because the smoothed idf below never reaches zero, `max_df` is what removes ubiquitous vocabulary.

$$\mathrm{tfidf} \;=\; \mathrm{tf} \times \left(\log\!\left(\frac{N+1}{\mathrm{df}+1} \right) + 1 \right)$$

The log is natural. The trailing $+1$ puts a floor under the idf: a term appearing in *every* document gets $\log(1) + 1 = 1$, not 0, and so keeps a non-zero score.

Documents as vectors. Each document is a point whose coordinates are its TF-IDF scores, one axis per term; most are zero, a *sparse* embedding. Euclidean distance is poor here, because a long document sits far from the origin and length swamps vocabulary. Measure the **angle** instead, which ignores length. Scores are never negative, so the angle runs from 0 (identical proportions) to $\pi/2 \approx 1.571$ (no shared vocabulary).

$$d(\mathbf{x}_a, \mathbf{x}_b) \;=\; \arccos\!\left(\frac{\mathbf{x}_a \cdot \mathbf{x}_b}{\|\mathbf{x}_a\| \, \|\mathbf{x}_b\|} \right)$$

- `DSText.compute_distances(anno, min_df=0.0, max_df=1.0)` — the angle distance for every pair. Each unordered pair appears *once*, with `doc_id` alphabetically before `doc_id2`.
- `anno.pipe(umap_text, doc_id=c.doc_id, term_id=c.lemma, n_components=2)` followed by `.predict(full=True)` — project the corpus down to two dimensions, returned as the columns `dr0` and `dr1`, one row per document. Takes `min_df` and `max_df` as well.
- Two corpora in different languages have different vocabularies, so their vectors live in different spaces and cannot be compared directly. Their *structure* can be.